Nagar YuwakShikshanSanstha's
YeshwantraoChavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: *To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration*	**Mission:** *To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

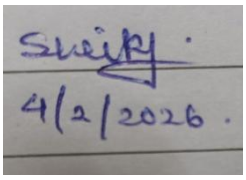
Keywords of POs:
Engineering knowledge,Problem analysis, Design/development of solutions,Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World,Ethics, Individual and Collaborative Team work,Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)
Sheikh Wasimuddin



Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (EVEN)**	**Course Name**	**JAVA FSD Lab**
Semester	**6**	**Course Code**	**23IOT1607**
Roll No	**63**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**2**
Course Outcome	1. Apply OOP concepts to solve the real life problem. 2. Use the database concepts to integrate in giving OOP oriented solution. 3. Develop full-stack applications integrating frontend, backend, and database with security. 4. Design and develop mini-project with frontend, backend, and database integration, including authentication and testing.
Aim	Apply the collection framework to implement the solution: • Arraylist • Sets • Maps
Problem Definition	Design and develop a system to manage high-volume logistics operations with the following requirements: • **Unique Tracking (Set):** Ensure that every package in the system has a unique Tracking ID. The system must prevent duplicate entries, even when packages are scanned at multiple checkpoints. • **Ordered Processing (ArrayList):** Maintain a complete historical log of each shipment's movements (checkpoints) in the exact order in which they occur, enabling accurate audit trails and traceability. • **Fast Lookup (Map):** Associate each Warehouse ID with its current inventory levels, mapping **Product Name → Quantity** to allow near-instant stock retrieval and updates.
Theory (100 words)	An **ArrayList** works like a diary by storing shipping locations in the exact order in which they occur. Each new checkpoint is added to the end of the list, and this operation is very fast because the

	system simply places the entry in the next available position without disturbing existing data. This makes insertion at the end an **O(1)** operation and is ideal for maintaining shipment history and audit records. A **Set** functions like a guest list by ensuring that no two packages have the same tracking ID. Before adding a new ID, the system checks whether it already exists. Using hashing, this check is performed in constant time, so duplicates are prevented efficiently without scanning the entire collection. Hence, insertion and lookup in a Set typically take **O(1)** time. A **Map** behaves like a dictionary by associating a warehouse ID with its stock levels. Each key (warehouse ID) directly points to its corresponding value (product name and quantity). This allows instant access to inventory data even when managing millions of records, as the system retrieves information directly using the key instead of searching through all warehouses. Lookup operations in a Map are therefore **O(1)**. In short, **O(1)** means constant time complexity, where the time required to perform an operation remains almost the same regardless of the size of the data set.
Procedure and Execution (100 Words)	**Step 1:** Start the program and initialize a HashSet<String> to store unique tracking IDs. **Step 2:** Initialize a HashMap<String, Map<String, Integer>> to manage warehouses. The key is the WarehouseID, and the value is another Map containing ProductNames and their StockCounts. **Step 3:** Use an ArrayList<String> within a Shipment object to store a sequential list of transit cities. **Step 4 (Validation):** Before adding a new shipment, check the **Set**. If the Tracking ID already exists, reject the entry as a duplicate. **Step 5 (Inventory Update):** When a shipment arrives at a warehouse, update the **Map**. If the product exists, increment the value; if not, add a new entry. **Step 6 (Audit):** Traverse the **ArrayList** to display the complete journey of a package from origin to destination. **Step 7:** Stop the program

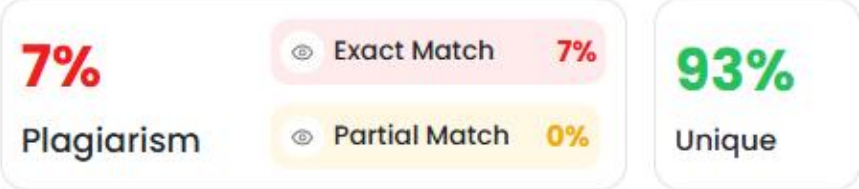
Code:

```
import java.util.*;

public class GlobalLogisticsSystem {

    public static void main(String[] args) {

        // Step 1: Set for unique tracking IDs
        HashSet<String> trackingIds = new HashSet<>();

        // Step 2: Map for warehouse inventory
        HashMap<String, Map<String, Integer>> warehouses = new
HashMap<>();

        String trackingId = "23071044_IoT63";

        // Check for duplicate tracking ID
        if (trackingIds.contains(trackingId)) {
            System.out.println("Duplicate Tracking ID! Shipment
rejected.");
            return;
        } else {
            trackingIds.add(trackingId);
            System.out.println("Shipment accepted with Tracking ID: "
+ trackingId);
        }

        // Create shipment and add checkpoints
        Shipment shipment = new Shipment(trackingId);
        shipment.addCheckpoint("Bilaspur");
        shipment.addCheckpoint("Raipur");
        shipment.addCheckpoint("Nagpur");
        shipment.addCheckpoint("Delhi");

        // Warehouse inventory handling
        String warehouseId = "WH-DELHI";
        warehouses.putIfAbsent(warehouseId, new HashMap<>());

        Map<String, Integer> inventory =
warehouses.get(warehouseId);

        // Adding products
        inventory.put("Laptops", inventory.getOrDefault("Laptops", 0)
+ 50);
```

```
        inventory.put("Monitors", inventory.getOrDefault("Monitors",
0) + 30);
        inventory.put("Mouse", inventory.getOrDefault("Mouse", 0) +
200);

        System.out.println("Inventory updated for warehouse: " +
warehouseId);

        // Display shipment journey
        System.out.println("\n--- Shipment Audit Trail ---");
        shipment.showJourney();

        // Display warehouse inventory
        System.out.println("\n--- Warehouse Inventory ---");
        for (String wh : warehouses.keySet()) {
            System.out.println("Warehouse: " + wh);
            for (Map.Entry<String, Integer> entry :
warehouses.get(wh).entrySet()) {
                System.out.println(" Product: " + entry.getKey() +
                        ", Quantity: " + entry.getValue());
            }
        }
    }
}
```

Output:
+

	Shipment accepted with Tracking ID: 23071044_IoT Inventory updated for warehouse: WH-DELHI --- Shipment Audit Trail --- Journey for Tracking ID: 23071044_IoT63 -> Bilaspur -> Raipur -> Nagpur -> Delhi --- Warehouse Inventory --- Warehouse: WH-DELHI Product: Mouse, Quantity: 200 Product: Monitors, Quantity: 30 Product: Laptops, Quantity: 50
Output Analysis	The output confirms that the logistics system is functioning correctly across tracking, shipment history, and inventory management. First, the system successfully **accepts a shipment with a unique Tracking ID (23071044_IoT63)**, indicating that duplicate tracking entries are prevented. Next, the inventory is **updated for warehouse WH-DELHI**, showing that stock changes are immediately reflected in the correct warehouse. The **Shipment Audit Trail** section displays the complete journey of the shipment in the **exact order of movement**: Bilaspur → Raipur → Nagpur → Delhi. This validates the use of an ordered structure (like an ArrayList) to maintain checkpoint history accurately for auditing and traceability purposes. Finally, the **Warehouse Inventory** section lists the current stock levels of WH-DELHI, showing products mapped to their quantities (Mouse: 200, Monitors: 30, Laptops: 50). This demonstrates fast and direct access to inventory data using a key-value structure (Map), ensuring efficient retrieval even in large-scale systems. Overall, the output verifies **unique tracking (Set behavior)**, **ordered shipment logging**, and **instant inventory lookup**, meeting all the core requirements of the logistics and inventory hub system.
Link of student Github profile where lab assignment has been uploaded	https://github.com/sheikhwasimuddin

Conclusion	The system successfully fulfills the core requirements of high-volume logistics management by ensuring unique identification of shipments, maintaining an ordered record of shipment movement, and enabling fast access to warehouse inventory data. The use of appropriate data structures improves efficiency, accuracy, and scalability, making the system reliable for real-time tracking, auditing, and inventory control in large logistics networks.
Plag Report (Similarity index < 12%)	
Date	**04-02-26**

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Paste here Faculty signed observation page.

PRACTICAL No: 02

Aim:- Apply the concept of collections framework & implement the solutions.
① Arraylist ② sets ③ Maps.

Problem statements: Global logistics & Inventory Hub develop a system to manage high volume logistics with following requirements: 4/2/26

① Unique Tracking (sets): Ensure every package in system has unique tracking ID.

② Ordered Processing (Arraylist): Maintain a historical log of every shipment movement in exact order they occured for audit trails

③ fast lookup (Maps): Associate each warehouse ID with its current stock levels for near-instant retrieval.

output:- shipment accepted with Tracking ID: 23071044 - IOT63
Inventory updated for warehouse : WH-DEL-H1
--- shipment Audit Trail ---

Journey for Tracking ID: 23071044 - IOT63
→ Bilaspur
→ Raipur
→ Nagpur
→ Delhi

--- Warehouse Inventory ---
warehouse : WH-Delhi
product : Mouse, Quantity : 200
product : Monitors, Quantity : 30
product : Laptops, Quantity : 50.

Shrikrupa